

Leant2

Native Dependent Program Synthesis in Lean

Architecture, algorithms, trust boundaries,
and executable experiments

Research and engineering design

September 17, 2026

Abstract

Leant2 should not be a transliteration of a Haskell inhabitation engine. It should be a native Lean elaboration-and-search system that constructs a program together with its behavioral evidence, retaining dependent types, universe levels, local instance identities, and metavariable dependencies throughout search. This article specifies such a system: a transactional native goal forest; bidirectional introductions, application-spine search, and dependent elimination; indexed component retrieval; fair portfolio scheduling; structural and well-founded recursive sketches; specification-directed branch synthesis; counterexample-guided finite backends; and kernel-checked publication under an explicit axiom and executability policy.

The central engineering separation is between semantic authority, search representation, and operational execution. Native expressions are authoritative, but open expressions alone are not portable search states. A proof-producing tactic may establish a type without supplying the desired behavior; a kernel-accepted recursor term may fail code generation; finite observational equality is not a safe permanent quotient; and exhaustion of a propositional or bounded synthesis fragment is not a proof that a Lean type is empty. Each boundary receives a concrete protocol and a precisely scoped guarantee.

Two included Lean-only prototypes were compiled remotely with Lean 4.34.0. One exercises dependent synthesis, shared-goal rollback, higher-rank callbacks, indexed elimination, and author-supplied recursive sketches. The other synthesizes an eight-node Boolean XOR expression using five counterexample-guided proposals and proves its universal Boolean specification by kernel reduction. These experiments validate narrow mechanisms, not the complete architecture or a broad coverage claim. The article supplies implementable algorithms, a module plan, benchmark gates, and a staged path to practical coverage without importing Haskell’s type restrictions into the successor.

Status. Architecture and production APIs are proposals. The two files in `prototype/` are actual checked experiments. No complete Leant2 implementation or comparative benchmark is claimed. Repository observations are pinned to Leant `6bf05ad78c46` and Djex `e8778f4ebd63`; Leant’s documented vendored Djex pin is separately `e237e866`.

Contents

1	Recommendation and design objectives	5
1.1	What changes relative to Leant and Djex	5
1.2	Goals that compete with each other	5
2	The request, its meaning, and its result	6
2.1	A synthesis problem in native type theory	6
2.2	Programs, examples, and laws	6
2.3	A result algebra, not a Boolean flag	7
2.4	Frontend elaboration protocol	7
3	Architecture and semantic authority	8
3.1	Three representations with different jobs	8
3.2	Worker ownership and the trust boundary	9
3.3	The origin record	9
4	Native Lean features that determine the design	10
4.1	Universes and higher-rank arguments	10
4.2	Dependency is more than an argument order	10
4.3	Data, propositions, and elimination	10
4.4	Class evidence is a value	10
5	The native search state and transactional kernel	11
5.1	A goal forest, not independent subproblems	11
5.2	A concrete backtracking failure	12
5.3	Snapshots and owned mutations	12
5.4	Parallelism requires an independence test	12
6	Bidirectional construction and focused search	13
6.1	The core judgment	13
6.2	Introduction rules	13
6.3	Elimination and context saturation	13
6.4	What is actually safe to focus	14
7	Application spines, type invention, and component retrieval	14
7.1	Spine search in detail	14
7.2	A practical type-invention ladder	15
7.3	Library indexing without semantic erasure	15
7.4	Typed abstract graphs as an accelerator	15
8	Scheduling a practical search portfolio	16
8.1	Lanes and work quanta	16
8.2	Fairness without pretending heuristics are optimal	16
8.3	Verification needs its own queue	16
9	Memoization, equivalence, and safe pruning	17
9.1	What belongs in a cache key	17
9.2	Definitional and propositional equality	17
9.3	Observational equivalence is provisional	17
9.4	Abstract interpretation with proof obligations	17
10	Recursive synthesis: schemes before unrestricted recursion	18

10.1	Why native types are not sufficient	18
10.2	The recursive sketch object	18
10.3	A constructive pointwise specification transform	18
10.4	Worked synthesis: relational list map	19
10.5	Worked synthesis: indexed vectors	20
10.6	Carrier invention: list indexing as a fold	20
10.7	Accumulator invariants and well-founded recursion	21
11	The kernel/compiler boundary	21
11.1	A boundary established by experiment	21
11.2	A two-stage lowering protocol	21
11.3	Partial and noncomputable definitions	22
12	Behavioral synthesis as coupled construction and proof	22
12.1	Three sources of useful constraints	22
12.2	Proof automation as a portfolio	23
12.3	Counterexample-guided synthesis with an unknown branch	23
12.4	Finite domains and a genuine universal result	23
12.5	External solvers and arithmetic	24
12.6	Higher-order observations and effects	24
13	Guarantees, limits, and negative evidence	24
13.1	The success guarantee is small and strong	24
13.2	Relative completeness is not a marketing claim	25
13.3	Three different meanings of “no solution”	25
13.4	Sound rejection from abstractions	26
13.5	Costs and minimality	26
14	Production modules and APIs	26
14.1	Module boundaries	26
14.2	Capability-oriented plugin interfaces	27
14.3	Native API adapters to implement first	28
14.4	Do not overload a tactic interface with declaration synthesis	28
15	Publication, provenance, and reproducible output	28
15.1	What the final receipt records	28
15.2	Exact-source replay	28
15.3	Atomic environment mutation	29
16	Operational safety and resource behavior	29
17	Practical coverage and deliberate boundaries	29
18	Executed Lean experiments	30
18.1	Method and reproducibility boundary	30
18.2	Native search: what was exercised	31
18.3	Finite CEGIS: exact grammar and trace	32
18.4	Failed attempts are part of the evidence	33
19	Benchmark plan and development gates	33
19.1	A benchmark that measures the intended system	33
19.2	Metrics and ablations	33
19.3	Milestone gates	34

20 Final design choices	34
A Artifact inventory and rerunning the experiments	34
B A compact implementation checklist	35

1 Recommendation and design objectives

Build Leant2 as a *Lean-native, certificate-producing program synthesizer*, implemented primarily as a library inside a project-matched Lean worker. Provide a tactic, a command interface, and a small Lean controller or REPL over the same request API. Its main engine should operate on native dependent goals, not on a translated simply typed or Haskell-like approximation. Retain specialized logical and finite synthesis engines as accelerators behind checked adapters, rather than making any one of their languages the universal intermediate representation.

A successful constrained request should return three distinct things: an implementation, a proof of the exact requested property, and a report identifying the assumptions and execution guarantees. These are not three spellings of “success.” A term can inhabit its type while violating the intended behavior. A term can satisfy a logical specification while being noncomputable. A compiled program can pass examples without satisfying a universally quantified specification. Leant2’s public result format must make these differences impossible to overlook.

The recommended initial target is ordinary functional programming with rich types: higher-order composition; records and sum types; subtypes and dependent pairs; finite indexed data; operations assembled from existing APIs; and total structural recursion over lists, trees, and user-defined inductives. These families share a manageable collection of synthesis rules. Arbitrary theorem discovery, arbitrary invariant discovery, quotient representative selection, and unconstrained type-level programming should be supported through additional strategies and explicit sketches, not promised as uniformly automatic capabilities.

1.1 What changes relative to Leant and Djex

The present systems already do substantially more than elementary arrow-type inhabitation. Djex combines a Djinn-derived logical engine with Exference’s ranked expression search; its current frontend and checking machinery retain higher-rank quantification, selected impredicative instantiations, and contextual evidence. Leant checks reconstructed Lean candidates and records exact source ownership. Those achievements should be preserved, not dismissed as a trivial Haskell prototype [1, 2, 4].

The concrete architectural bottleneck is visible in Leant’s `Synth/Fragment.hs`. Its Lean-side serializer decomposes supported native expressions into a fragment consumed by the Haskell engines. The implementation explicitly treats term-indexed applications as opaque and restricts structural expansion of inductives. It also retains additional universe and contextual metadata to reconstruct cases that its simpler search representation cannot directly express. Leant2 removes this *mandatory semantic translation* while keeping optional abstractions for indexing and specialized search [3].

There is already a repository report recommending a semantic Lean rewrite, including native expressions, a project-local worker, and explicit checking boundaries. This article takes that feasibility conclusion as a starting point. Its additional contribution is an algorithmic design for native dependency-aware search, contract-driven recursive synthesis, a compiler-lowering boundary, and concrete experiments. It does not claim that moving an implementation language alone solves search complexity [6].

1.2 Goals that compete with each other

There is no single objective called “best synthesis.” A useful system must balance time to first valid result, quality of the result, diversity of alternatives, proof cost, memory use, and predictable behavior under a resource limit. The shortest term is frequently an unwanted default. The fastest successful tactic may introduce a noncomputable choice. A normal form suitable for a completeness proof may be a poor executable program.

Therefore requests should name a *profile*. A practical default prioritizes executable total programs, inexpensive library reuse, and small proof obligations. A proof profile prioritizes theorem closure

Concern	Preserve from the predecessors	Change in Leant2
Semantic identity	Exact candidate ownership and source inventory	Native <code>Expr</code> , levels, names, and contexts throughout
Search	Logical focusing and ranked alternatives	One dependency-aware goal forest, plus specialized lanes
Behavior	Exact proposition checking and inconclusive outcomes	Push contracts into construction, recursion, and counterexample refinement
Negative answers	Conservative completeness accounting	Distinguish certified impossibility from every form of fragment exhaustion
Output	Independently checked displayed candidates	Frozen declarations, separate executability check, replayable source
Compatibility	Useful Lean behavior and diagnostics	No Haskell frontend, kind language, or legacy API obligations

Table 1: A semantic redesign, not a module-by-module port.

and permits a declared classical foundation. A pedagogy profile prefers readable constructions. An exhaustive-fragment profile fixes a grammar and reports its exact search boundary. Profiles change priorities and explicit admissibility policies; they must not silently change the requested type or property.

2 The request, its meaning, and its result

2.1 A synthesis problem in native type theory

Fix a Lean environment $\mathcal{E}$, a well-formed local context Γ , an expected type T , and a behavioral predicate Φ . The core obligations are

$$\mathcal{E}; \Gamma \vdash T : \text{Sort}(u), \quad \mathcal{E}; \Gamma \vdash \Phi : T \rightarrow \text{Prop}.$$

A verified solution consists of expressions t, p such that

$$\mathcal{E}; \Gamma \vdash t : T, \quad \mathcal{E}; \Gamma \vdash p : \Phi(t). \tag{1}$$

Without a behavioral constraint, take $\Phi(t) = \text{True}$. The request also includes an admissible component inventory, an axiom policy, an execution policy, and search budgets. These determine what the system is allowed to return, not what the bare type means.

For $T : \text{Type}(u)$, one can package a result as a subtype $\{t : T \mid \Phi(t)\}$. The public metaprogramming API should instead retain separate expression fields, or use a genuinely sort-polymorphic dependent packaging such as `PSigma`. Using `Subtype` as though it accepted an arbitrary $T : \text{Sort}(u)$ would unnecessarily restrict proof-valued and sort-polymorphic requests.

Local assumptions remain local assumptions. When a result is exported, close over the required telescope in dependency order. A candidate obtained under $h : \text{False}$ is conditionally valid but says nothing about an unconditional executable implementation. An acceptance report should therefore display the abstracted premises and the transitive axiom inventory, rather than presenting a closed-looking name that conceals its context.

2.2 Programs, examples, and laws

Consider three requests for a binary selector:

```
-- Proposed production syntax, not the prototype's parser.
synth def choose : (A : Type) → A → A → A

synth def choose : (A : Type) → A → A → A
  satisfying choose Nat 11 29 = 29

synth def choose : (A : Type) → A → A → A
  satisfying ∀ A a b, choose A a b = b
```

The first merely requests an inhabitant. The second proves one observation. The third proves a law. Parametricity may suggest that the second observation is highly discriminating within a particular pure grammar, but Leant2 should not silently invoke a parametricity theorem that it has not established for the actual grammar and environment. It should prove the third property separately when that is the desired guarantee.

The native prototype does precisely this separation: it synthesizes a selector satisfying the finite observation and then proves a stronger theorem about the particular generated term by reflexivity. That stronger theorem is additional evidence; it is not a reinterpretation of the user's finite example.

2.3 A result algebra, not a Boolean flag

Result	Required evidence and permissible interpretation
Verified	Exact $t : T$ and $p : \Phi(t)$ accepted under the declared axiom policy. Execution status is a separate field.
TypedOnly	$t : T$ checked, but the requested contract has not been proved. Never silently promoted to verified.
TestedOnly	Typed candidate with recorded finite observations. Useful for exploration, not proof of an unstated law.
CandidateRefuted	A checked proof of $\neg\Phi(t)$, or a checked counterexample sufficient to derive it. This rejects this candidate only.
Impossible	A checked term of $(t : T) \rightarrow \Phi(t) \rightarrow \text{False}$, relative to the displayed context.
FragmentExhausted	A precise finite grammar or certified decision fragment has been exhausted. The claim is about that fragment, not all Lean terms.
BudgetExhausted	Search or verification was interrupted by its resource limit. Include remaining queues and resumable state identifiers.
Unsupported	A requested policy or backend cannot handle an obligation. Other lanes may still work; unsupported is not false.
InfrastructureError	Worker failure, version mismatch, transport error, or failed replay. This conveys no mathematical conclusion.

A user interface may group several of these states, but its structured API must preserve them. A complete candidate can be simultaneously `TypedOnly`, refuted by one example, and useful as a source of reusable subterms. Search progress and final authorization are separate concerns.

2.4 Frontend elaboration protocol

The frontend first elaborates T in the user's current namespace and scope. It then introduces a fresh local name of type T and elaborates the entire constraint as a proposition. Unknown names should be errors rather than newly invented implicit parameters. Any expected universes and local instances are captured

before search begins. This follows the good preflight principle already used by Leant’s named behavioral entrance, but retains the resulting native expressions directly [5].

The synthesizer distinguishes *rigid* user parameters from *owned* search metavariables. It must not solve a request by assigning an ambient user metavariable that the caller intended to remain independent. The safe default is to freeze external metavariables, or require an explicit elaborator contract stating which assignments may escape. A tactic invoked inside an unfinished proof must return its permitted assignments transactionally to that proof state.

The frontend offers three views over one service: by `leant2` fills an expected term; a `##synth?` command explores alternatives without registering a permanent definition; and a declaration command publishes a named implementation plus a named correctness theorem atomically. The REPL should be an adapter, not a second elaborator.

3 Architecture and semantic authority

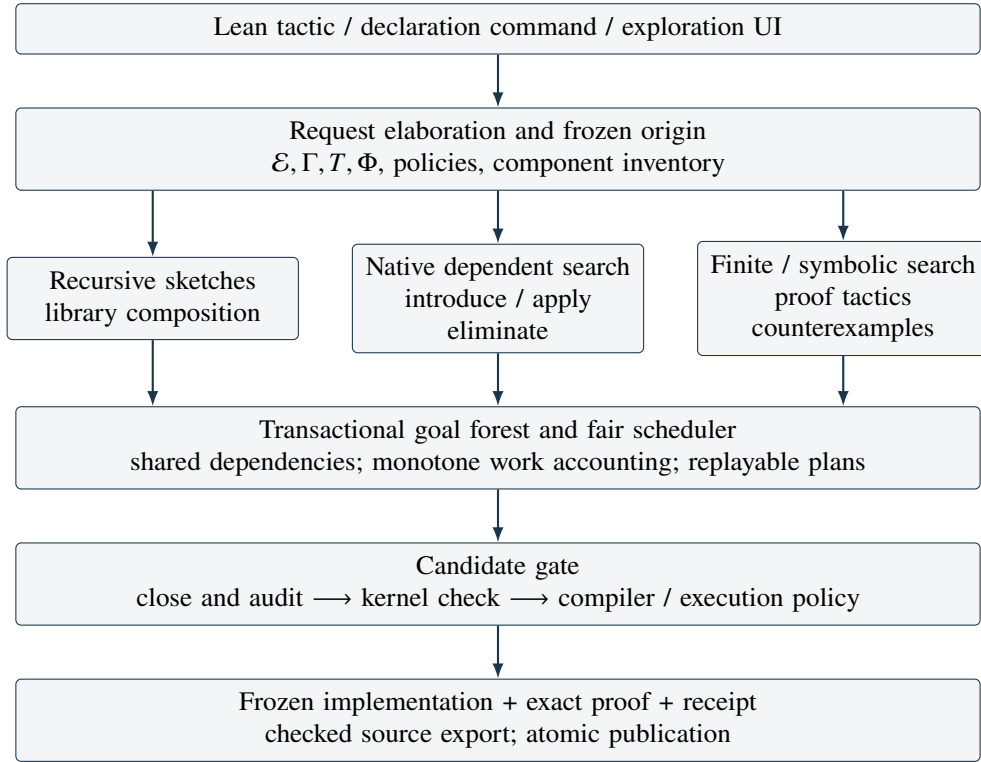


Figure 1: All proposal lanes share one native semantic boundary. Specialized search languages are optional adapters, never authorities for the whole problem.

3.1 Three representations with different jobs

Semantic representation. Native Expr, Level, Name, binder information, local contexts, local instances, and the environment identify what a goal or candidate means. Preserve nominal constants and exact local dictionary expressions. A pretty-printed string is not a substitute for any of these.

Search-plan representation. A persistent plan records introductions, applications, constructor choices, case splits, recursive schemes, and proof obligations. It may contain native expression annotations closed over a recorded telescope. It exists to resume and explain search, to lower recursive definitions, and to share immutable structure. It must not quietly introduce a competing typing relation.

Acceleration representations. Discrimination-tree keys, type-head graphs, abstract lengths, finite Boolean syntax, and solver formulas deliberately omit information. Each comes with a reconstruction or soundness boundary. A coarse index proposes a component; native unification decides whether that use is viable. A solver proposes a witness or certificate; Lean determines whether it proves the original statement.

This division reconciles two desirable properties that otherwise seem to conflict: retaining genuine Lean types and keeping search state serializable. Serialize a closed, scoped plan and its origin; do not serialize a naked open `Expr` and assume that its free-variable identifiers retain their meaning in another worker.

3.2 Worker ownership and the trust boundary

Use the project’s selected toolchain to run the worker. Its environment owns imported declarations, namespace resolution, instance search, and the exact Lean API version. An editor integration may invoke the library in the server process for responsiveness, while a strict isolation profile delegates expensive speculative work to a restartable subprocess. The standalone controller and worker can both be Lean programs. No Haskell service is required.

“Implemented in Lean” need not mean reimplementing the operating system or the Lean runtime. A hardened deployment can retain a small platform-specific process-launch layer. The synthesis algorithms, policies, protocol, and certificate construction remain Lean code. An external SMT solver is optional, untrusted, and replaceable. The no-solver profile must still provide useful native synthesis.

Within a worker, a plug-in is not automatically safe merely because its result will eventually be checked. A plug-in can mutate the environment, create new axioms, change options, execute arbitrary IO, or conceal dependencies through generated helpers. Speculation therefore runs against an origin snapshot with a restricted publication interface. Only a candidate gate may commit new user-visible declarations. In the strongest profile, replay occurs in a fresh process containing the declared imports and candidate bundle, but no speculative state.

3.3 The origin record

An immutable `Origin` should include a toolchain identity, imported module identities, an environment revision, the closed local telescope, T , Φ , component policy, axiom policy, execution policy, and semantic options. The component inventory records constants and local terms, not their display names alone. It also records prohibited providers and whether that prohibition extends transitively to implementation dependencies.

Use fingerprints to find possible cache hits, not as substitutes for structural validation. An in-process cache can exploit persistent environment identity. A cross-process cache needs a canonical serialized form, dependency fingerprints, and collision-resistant digests. A hit must still be replayed against the current origin before it becomes an authorized answer.

Options that affect elaboration or search semantics belong in the origin: transparency policy, active local instances, coercion configuration, admissible classical assumptions, and grammar restrictions. Formatting preferences need not invalidate a semantic candidate, but should invalidate a cached presentation artifact when they change its spelling.

4 Native Lean features that determine the design

4.1 Universes and higher-rank arguments

Lean’s `Prop` is impredicative; its `Type` hierarchy is not. A callback that accepts a polymorphic function is therefore not permission to instantiate every type metavariable by every polymorphic expression. Native level constraints must remain attached to the whole application spine. In particular, the polymorphic identity type over `Type` does not itself inhabit `Type`; the negative universe control in the experiment checks this exact mistake [7, 8].

For a provider with interleaved type, term, implicit, and instance binders, open the telescope in order, creating fresh owned metavariables with their proper local contexts. Later arguments may constrain earlier type choices. A single occurrence must share those assignments throughout its spine. Distinct provider occurrences normally receive fresh instantiations. Global universe parameters and lexical type variables remain rigid.

Native representation removes artificial fixed limits such as “at most six correlated type binders.” It does not eliminate the cost of discovering a useful instantiation. Leant2 should expose budgets for type-expression invention, application depth, and carrier complexity, not encode a semantic rank ceiling into its source language.

4.2 Dependency is more than an argument order

For a constructor

$$c : (a : A) \rightarrow B(a) \rightarrow C(a),$$

choosing a creates obligations whose types mention that exact choice. For a subtype constructor, choosing a value creates a proof goal about that value. For an indexed constructor, the result’s index equation may solve an earlier metavariable, rule out the constructor, or remain blocked. These are native dependency constraints, not independent slots in a product.

The environment also contains dependencies that are easy to miss: an instance can depend on a type metavariable; a coercion can expose an instance obligation; a local declaration can refer to another declaration’s value; a postponed elaborator task can later assign a shared hole. The scheduler’s dependency graph must include the transitive closure of such relations, not just the syntactic free metavariables in the displayed targets.

4.3 Data, propositions, and elimination

An inhabitation proof `Nonempty A` is not generally an executable element of `A`. Similarly, a proof of a disjunction is not a freely inspectable runtime sum. The allowed eliminator depends on the inductive declaration and its sort. Use Lean’s actual recursor metadata and elaborator APIs; do not derive a runtime case split by treating every logical connective as a data constructor [9].

Equality transport is a first-class synthesis operation. A request for $B(b)$ can be solved from $v : B(a)$ and $h : a = b$ by a checked transport. Dependent case analysis should generalize or revert the context that depends on the scrutinee, then reconstruct it in each branch. Trying to substitute text into a nondependent pattern matcher is not a reliable implementation of this operation.

4.4 Class evidence is a value

Lean’s native instance synthesizer should resolve ordinary instance-implicit obligations using the active lexical instances and standard search policy. It already handles the language’s output-parameter conventions. Leant2 should not duplicate that machinery in a Haskell-style dictionary approximation [10].

Nevertheless, two explicitly supplied structures of the same class type need not have equal computational fields. The search inventory must preserve both terms. Deduplicating providers because their types have the same printed predicate loses real implementations. The correct boundary is: ordinary inference follows Lean’s implicit-instance behavior; explicit evidence remains an ordinary selectable value. Switching between these policies is a user-visible choice, not an optimization.

The prototype’s `exactDictionary` test uses two explicitly supplied records to exercise identity preservation. It does not implement or benchmark a new instance-resolution algorithm.

5 The native search state and transactional kernel

5.1 A goal forest, not independent subproblems

Represent a live branch by

$$S = (O, \mathcal{M}, \Gamma_\bullet, G, D, P, C, H),$$

where O is the immutable origin, $\mathcal{M}$ the branch’s metavariable state, $\Gamma_\bullet$ the contexts associated with its goals, G the residual goal forest, D the dependency graph, P the construction plan, C the constraint store, and H the branch history. The branch also has a scheduling priority, but the global work ledger belongs outside this rollbackable record.

A goal node is an owned `MVarId` with a role: implementation, behavioral proof, type argument, instance, equality transport, termination proof, or auxiliary lemma. A role guides scheduling; it does not change the goal’s native type. Edges in D record dependencies through metavariables and local declarations, including dependencies exposed by postponed elaboration. Every state also maintains a set of external metavariables that may not be assigned.

The first implementation can use persistent Lean snapshots rather than a custom substitution engine. The important operation is an *atomic attempt*: restore a checkpoint, try a native action, and commit only if the whole relevant continuation succeeds. No result from a failed continuation may remain in the parent metacontext.

Algorithm 1 Dependency-preserving depth-bounded reference search

```

1: function SOLVEFOREST( $G, b$ )
2:   Remove already assigned goals from the front of  $G$ 
3:   if  $G = \emptyset$  then
4:     return success
5:   end if
6:   if  $b = 0$  then
7:     return bounded failure
8:   end if
9:   Let  $g$  be the first goal and  $R$  the remaining goals
10:  for each applicable action proposal  $a$  for  $g$  do
11:     $s \leftarrow$  snapshot of the branch-local semantic state
12:    Charge the global work ledger for attempting  $a$ 
13:    Run  $a$  in  $g$ ’s context, obtaining children  $G_a$ 
14:    if the action succeeded and SOLVEFOREST( $G_a \mathbin{++} R, b - 1$ ) succeeds then
15:      return success
16:    end if
17:    Restore  $s$ ; retain only tagged diagnostics and work charges
18:  end for
19:  return bounded failure
20: end function

```

Algorithm 1 is intentionally a reference algorithm, not the production scheduler. Its merit is its continuation discipline. Solving the first child and irrevocably committing its first answer is wrong whenever a later child constrains that answer. The reference prototype implements this pattern with native `Meta.saveState`, `MVarId.apply`, `intro1P`, and `cases`, all checked against Lean 4.34.0 in the supplied experiment.

5.2 A concrete backtracking failure

Suppose the root asks for

$$\{f : (A : \text{Type}) \rightarrow A \rightarrow A \rightarrow A \mid f \mathbb{N} 11\ 29 = 29\}.$$

Constructing the subtype opens a value goal $?f$ and a proof goal depending on it. The first plausible value, $\lambda A\ a\ b.\ a$, solves the value goal but changes the proof goal into $11 = 29$. That continuation fails. A correct search restores the value assignment and tries $\lambda A\ a\ b.\ b$, after which reflexivity closes the proof goal.

A solver that reports “the value subproblem is solved” and later treats the equality as an unrelated impossible subproblem misses the solution. The same problem occurs with dependent witnesses, type arguments inferred from later arguments, recursive accumulator types, and competing class dictionaries. The forest is not merely an optimization; it is the unit of semantic backtracking.

5.3 Snapshots and owned mutations

In the miniature experiment, actions operate in `MetaM`, and a `Meta.SavedState` is the relevant rollback mechanism. Production actions may invoke term elaboration, tactic elaboration, declaration elaboration, messages, caches, and environment extensions. A version-specific transaction adapter must capture and restore the corresponding states and reader contexts as well. Its contract should be tested through deliberate assignment, fresh-name, local-instance, message, and environment-mutation failures.

Do not swallow cancellation, out-of-memory failures, or global budget exceptions as ordinary “rule not applicable.” The prototype catches exceptions broadly for brevity; the production adapter needs typed outcomes such as `notApplicable`, `blocked`, `rejected`, `yielded`, and `aborted`. Local failure can trigger rollback. A hard cancellation must unwind to the controller and retire or reset the worker.

Avoid storing closures that accidentally capture mutable metavariable state in a supposedly persistent queue. A resumable task should contain an origin, a saved branch state or replayable plan, an action cursor, and enough information to reinstall its local context. Fresh metavariable identifiers created in different branches must never be merged merely because their numeric components match.

5.4 Parallelism requires an independence test

Two proof goals may be solved concurrently only when the worker can establish that their assignment footprints are disjoint, including transitive type and instance dependencies. A useful conservative policy initially parallelizes entire isolated lanes or completed candidate verifications, not arbitrary siblings. This loses some parallelism but keeps correctness and reproducibility tractable.

Later, use connected components of D as scheduling units. An independence certificate can record disjoint owned metavariable sets, disjoint synthetic obligations, and an unchanged shared rigid context. When either task introduces a new dependency, merge the components or serialize the conflicting step. Never make separate successful snapshots compete to overwrite the same metavariable state.

6 Bidirectional construction and focused search

6.1 The core judgment

Write

$$\Gamma \vdash ?m \Leftarrow T [\Phi]$$

for construction of a term at an expected native type, together with residual contract obligations. The implementation does not parse this judgment into a separate calculus; it uses the corresponding native goal. It is a useful notation for describing search rules.

The important distinction is between *introducing* the outer structure of an expected type and *using* a value already available in the context or component inventory. Introductory forms drastically reduce search when the expected type is known; elimination forms discover which existing function, projection, or case split can produce the desired result. They should meet in application-spine search rather than in an untyped enumeration followed by type checking.

6.2 Introduction rules

For a dependent function type, introduce a fresh rigid local and continue:

$$\frac{\Gamma, x : A \vdash b \Leftarrow B(x)}{\Gamma \vdash \lambda x. b \Leftarrow (x : A) \rightarrow B(x)}.$$

The original binder information must be preserved for export. Introducing an implicit argument is not permission to make it an explicit user-visible parameter, and a strict implicit binder should not be reconstructed from an ordinary implicit one by guesswork.

For a constructor c of a target inductive, instantiate its universe parameters freshly, open its telescope, and unify its result with the expected target. Keep the resulting argument holes and index constraints in one transaction. A constructor that cannot match is rejected; a constructor blocked on an unresolved parameter is suspended rather than certified impossible. Ordinary pairs and subtypes are special cases of this rule.

For a behavioral goal, construction should also produce a decomposition opportunity. If the requested relation has a constructor aligned with the output constructor, introduce both together. For example, proving a pointwise list relation exposes the output head and a proof relating it to the input head. This can be far more selective than first generating an arbitrary list and only then checking it.

6.3 Elimination and context saturation

Use local assumptions in four basic ways: exact matching, partial or full application, projection, and case analysis. Exact matching should precede gratuitous eta-expansion when the user's quality profile rewards direct reuse. Projections are cheap for structures but must remain typed at the original record value. Forward application can add useful derived terms, but it needs a cost bound and a relevance filter to avoid saturating the context with every possible application.

For a sum-like input, a case split creates a branch for each reachable constructor. Branch obligations share their outer parameters, and all must succeed. Dependent targets require an appropriate motive; delegate motive construction and index refinement to native case-analysis machinery. An indexed vector at length $n + 1$, for example, does not have an admissible empty branch merely because an approximate unindexed vector has an empty constructor.

Repeatedly splitting recursive inputs is a bounded exploration strategy, not a replacement for recursion synthesis. A depth-limited expansion of a list cannot implement an arbitrary-length traversal unless the result contains a genuine recursion scheme. Prefer a recursion lane once a recursive data input is structurally relevant and ordinary component use has stalled.

6.4 What is actually safe to focus

Logical proof search often applies provability-preserving rules eagerly. Aesop’s safe/unsafe distinction and AND–OR search organization are useful precedents, but its documented automation is a proof tactic, not a complete algorithm for inventing recursive programs [15, 16].

Leant2 needs a stronger classification. A transformation may preserve the existence of an inhabitant while changing the available implementations, their cost, or their satisfaction of a behavioral constraint. Replacing a direct function value by its eta-expansion is harmless for many extensional contracts but can change the search cost. Splitting a product goal is logically natural, but an existing provider might return the whole product more cheaply. Dropping an unused-looking input is not generally behavior-preserving.

Use three rule labels: *checked transformation*, *grammar-preserving normalization*, and *heuristic proposal*. The first means Lean can validate its effect; the second additionally carries a theorem or explicitly delimited argument about the covered grammar; the third promises only a useful candidate. Fast lanes may focus aggressively, but a fairness lane must retain alternatives lost by such focusing whenever a relative completeness guarantee is advertised.

7 Application spines, type invention, and component retrieval

7.1 Spine search in detail

Suppose a candidate head has native type

$$h : (a_1 : A_1) \rightarrow (a_2 : A_2(a_1)) \rightarrow \cdots \rightarrow (a_k : A_k(a_1, \dots, a_{k-1})) \rightarrow R(a_1, \dots, a_k).$$

At each prefix, attempt to match the remaining type against the expected goal. This admits returning a partially applied function; it is not restricted to providers whose fully saturated codomain matches the target. Keep a head occurrence’s complete argument vector, including implicit and instance arguments, because later choices may refer to any earlier entry.

Algorithm 2 Demand-directed native application-spine search

```

1: function SPINES( $h, T, \Gamma, b$ )
2:   Freshen the global universe parameters of this occurrence of  $h$ 
3:   Set  $e \leftarrow h$  and infer its native type  $U$ 
4:   for  $j = 0, 1, \dots, b$  do
5:     In a fresh transaction, try native unification of  $U$  with  $T$ 
6:     If successful, emit the partial plan  $e$  and all residual obligations
7:     Reduce  $U$  only enough to expose its next binder
8:     if  $U$  is not a dependent function type then
9:       return
10:    end if
11:    Create an owned argument hole  $a_j$  at the binder’s exact domain
12:    Classify the hole as term, type, instance, or postponed elaboration
13:    Set  $e \leftarrow e a_j$ ; substitute  $a_j$  into the remaining type  $U$ 
14:    Propagate resulting constraints without committing a final choice
15:  end for
16: end function
  
```

In practice the native `apply` family already performs much of the initial matching. The production adapter adds resumable alternative instantiations, explicit argument provenance, and failure classification rather than replacing Lean’s unifier [11]. The exact prototype’s `apply` calls exercise only a bounded subset of this proposed functionality.

An unsuccessful higher-order unification attempt is not a proof of disequality. Some constraints are syntactic conflicts; others require a currently unknown term or a non-pattern instantiation. Classify a blocked constraint by the metavariables that could unblock it and place a watcher on them. Retry after those variables change. A budget or depth exception belongs to the scheduler, not in a permanent negative cache.

7.2 A practical type-invention ladder

When native inference cannot determine a type-valued argument, enumerate increasingly expressive candidate types. Start with exact types already present in the query telescope, output type, and contract. Then use types of local values and relevant component domains. Next construct small products, sums, indexed applications, and function spaces from those seeds. Admit quantified types and accumulator carriers at later levels. Keep a separate size cost for these invented type expressions.

For example, a list fold used for indexed lookup may need an accumulator of type $\mathbb{N} \rightarrow \text{Option}(A)$ rather than $\text{Option}(A)$. Looking only at the final output type misses the useful carrier. A demand-directed rule should propose function carriers over arguments that are consumed *after* the structural traversal. Such rules are hypotheses about useful search spaces; each resulting type must still satisfy the exact native universe constraints.

Do not hard-code unrelated monotypes such as Nat as universal stand-ins for polymorphic variables. They can be admitted as literal component types when relevant, but a solution synthesized for one concrete type is not a solution to a universally quantified request. Type-variable names, lexical scopes, and universe levels remain part of the semantic problem.

7.3 Library indexing without semantic erasure

Maintain an index of admitted declarations, local providers, constructors, projections, and registered recursive combinators. A `DiscrTree`-style structure can organize normalized result heads, significant rigid arguments, and arities. The index key deliberately omits detail; every hit is freshly instantiated and checked by native matching before becoming a search action. Goals with unknown result heads need a fallback inventory walk or a widening tier, otherwise an index optimization silently becomes a coverage restriction.

The proposed query plan has four tiers: local values and constructors; explicitly named providers; relevant namespaces and registered component sets; and an optional wider environment search. Every request records which tiers were enabled. A limit of fifty retrieved providers is a useful heuristic bound, but an exhausted fifty-provider search cannot be reported as exhaustive over the imported environment.

A component record should expose both its native type and optional proved summaries: input/output relations, preservation laws, size relations, and executable status. Summaries reduce search only through checked implications. Documentation embeddings or learned relevance scores can rank hits but cannot establish that a component has a property it has not proved.

7.4 Typed abstract graphs as an accelerator

Type-guided abstraction refinement, as exemplified by TYGAR, motivates a scalable component-composition lane: search a graph over coarse type shapes, elaborate the proposed path natively, and refine the abstraction when a spurious composition fails [20]. In Leant2 this graph is advisory. It cannot assign away a dependent index or choose an inconsistent universe merely to connect two nodes.

A useful abstract key is a rigid type head plus a bounded pattern of its arguments. A failed path can teach the graph to distinguish, for example, $\text{Vec } A \ 0$ from $\text{Vec } A \ (n+1)$, or two different local dictionaries. Refine with an explainable native mismatch. Avoid a rule that globally bans a provider after one specialization fails: the same provider may work under another argument vector.

8 Scheduling a practical search portfolio

8.1 Lanes and work quanta

Use several lanes over the same origin: local focused synthesis, component composition, recursive sketches, proof automation, and specialized finite or symbolic synthesis. Each exposes a bounded-step interface returning a new continuation, zero or more candidate proposals, and a work delta. A proposal is not a success until it crosses the shared candidate gate.

A practical scheduler gives the fast native lane a larger initial share, but reserves a nonzero share for widening and recursive search. Rotate lanes after a finite work quantum, not only after each lane produces a candidate. An engine that spends indefinitely computing its next result otherwise starves the rest. Costly native primitives that cannot be suspended internally should have a per-attempt limit and be restarted at larger allowances in a later tier.

Global counters are monotone: attempted rule applications, generated candidates, solver calls, normalization steps where available, and verification work remain charged after rollback. Otherwise a branch can repeatedly fail and restore its budget. Memory limits count retained snapshots and continuation graphs, not just completed terms.

8.2 Fairness without pretending heuristics are optimal

Let a task have a lane number i , expression-cost bound s , recursive-scheme bound r , and verification allowance level v . A diagonal schedule visits finite tuples in increasing $i + s + r + v$, with deterministic tie breaking. Within a tuple, a heuristic queue may prioritize promising states. To preserve the fairness claim, reserve a FIFO stream that eventually visits every admissible state in that finite tier, or bound the number of heuristic overtakes.

Do not advertise a global shortest-program theorem for a best-first queue with arbitrary learned scores, retrieval caps, or unbounded proof costs. A precise optimality claim is possible only relative to a fixed grammar, a defined cost, complete enumeration below the returned cost, and sufficiently complete verification. Ordinary interactive output should instead say “best verified candidate found under this profile.”

A practical score can combine syntax size, case splits, invented type complexity, library distance, estimated proof effort, and deviation from supplied examples. Input usage is a soft preference. Lean is not a linear language; a valid implementation may ignore or duplicate arguments. Hard usage restrictions must be user-specified grammar constraints, not hidden assumptions inherited from a resource-sensitive search engine.

8.3 Verification needs its own queue

A candidate with a hard proof obligation should not block candidate generation. Assign each candidate a small verification quantum, retain inconclusive candidates, and enlarge their budgets fairly. A new counterexample can cheaply eliminate many waiting candidates. A new lemma can make several previously blocked proof obligations tractable. Both events should wake suspended work instead of restarting the whole search.

The user may request the first verified answer or several diverse alternatives. Diversity should distinguish actual native terms or proved equivalence classes, not just whitespace or pretty-printer choices. Two spellings of the same candidate do not earn independent successes, and two different typed owners do not exchange evidence merely because they print alike.

9 Memoization, equivalence, and safe pruning

9.1 What belongs in a cache key

A target type alone is not a synthesis subproblem. A cache key needs the relevant local telescope, instance evidence, native target, residual contract, origin identity, permitted components, and search restrictions. It also needs the assignment state of any shared metavariables, or a canonical abstraction that records their dependency relationships. Reusing a result proved under a stronger path condition in a weaker context is unsound.

A reusable success is best stored as a term abstracted over a canonical telescope, together with its type and proof. Instantiate this template in the destination context and validate it. A failed search result is reusable only with its exact bounded grammar and budgets, and only as evidence that that particular search was exhausted. Resource failure is not a logical conflict clause.

9.2 Definitional and propositional equality

Use native checks to identify definitional equality where inexpensive, with explicit transparency and rollback. For more aggressive rewriting, retain a checked equality and transport the contract along it. Homogeneous equality requires compatible types; a dependent expression graph cannot indiscriminately merge nodes at different fibers. Restrict an initial equality-saturation plug-in to closed monomorphic regions or to regions whose typed transport infrastructure is explicitly supplied.

Do not use a syntactic e-graph’s congruence closure as a replacement for Lean’s conversion checker. Even well-motivated normalization can change operational cost or cross an opacity boundary. Keep the original candidate available for execution-policy comparison, and recheck the final optimized candidate’s property rather than borrowing an untransported proof.

9.3 Observational equivalence is provisional

For examples $E = \{x_1, \dots, x_m\}$ define an observation signature

$$\text{obs}_E(t) = (t(x_1), \dots, t(x_m)).$$

Equal signatures are useful for ranking and batching, but are not proof of functional equality. As E grows, one bucket can split. Retain all representatives, a compressed version space, or enough derivations to regenerate the bucket when a new example arrives.

A minimal counterexample is a grammar containing only the two Boolean expressions x and y . On input $(0, 0)$ they have the same value. Keeping only x permanently destroys the solution to the specification “return y ” when the next input is $(1, 0)$. The supplied finite prototype includes checked lemmas witnessing an analogous collision and later separation.

For a *known finite complete domain*, an exact truth table can support permanent quotienting, provided the semantics is compositional for the allowed contexts and the equivalence is checked. That is a special finite backend theorem, not a general justification for discarding higher-order or dependent Lean programs after a few tests.

9.4 Abstract interpretation with proof obligations

A plug-in can associate a partial program P with an overapproximation $A(P)$ of all behaviors of its admissible completions. The sound pruning rule is

$$\text{Behaviors}(\text{Completions}(P)) \subseteq A(P), \quad A(P) \cap \text{Allowed} = \emptyset \implies \text{prune } P.$$

For example, a length domain can track intervals or affine relations for lists; a constructor-tag domain can track which output alternatives remain possible. The inclusion must be justified for every admitted component and path condition. An unknown component summary yields top, not an empty set.

A first implementation should use proof-carrying transfer summaries for a small library and replay rejection certificates in Lean. A more ambitious implementation can formalize the abstract domain and its soundness once. Neither permits declaring a source contract false merely because an approximate model has no solution under an accidental translation restriction.

10 Recursive synthesis: schemes before unrestricted recursion

10.1 Why native types are not sufficient

A native solver can compose callbacks and fill dependent constructor fields without automatically discovering an induction principle, an accumulator invariant, or a useful traversal order. Recursion must be an explicit search dimension. The recommended progression is: reuse an existing operation; instantiate a registered fold or traversal; generate a structural recursive sketch; then try well-founded, nested, or mutually recursive sketches with additional obligations.

This is related to the use of reusable recursive templates in synthesis research, but Lean changes the implementation boundary: a template is instantiated against an actual dependent telescope, and termination and behavioral obligations are checked in the same host logic [19]. The template library is a strategic restriction that improves feasibility, not a replacement for the source type system.

10.2 The recursive sketch object

A `RecSketch` contains a proposed function telescope, the selected decreasing argument or arguments, the result family, constructor branches, permitted recursive calls, and a lowering plan. A recursive-call capability records the smaller argument and the evidence making that call admissible. It is not an unrestricted local function of the final target type.

For ordinary structural recursion, the initial capabilities are the recursive results supplied by the selected recursor or by known immediate recursive subterms. Branch bodies can use those results, ordinary local fields, and admitted components. A branch cannot call the synthesized function on the unchanged original input unless a separate, accepted termination argument justifies it.

The sketch's result family should quantify over parameters that may change during recursion. For vector indexing, an index argument depends on the vector length; for accumulator algorithms, the accumulator changes between calls. Generalizing too little makes the induction hypothesis unusable. Generalizing everything creates larger higher-order goals. Enumerate small dependency-closed generalization sets, guided by variables appearing in recursive-call arguments and residual contracts.

Ordinary strictly positive inductives, indexed families, nested inductives, and mutual inductives do not have one interchangeable branch shape. The initial implementation should support a documented subset and use registered adapters for more elaborate recursors. Native metadata establishes which eliminators exist; it does not by itself tell a heuristic which motive will make a program easy to synthesize.

10.3 A constructive pointwise specification transform

Suppose a contract has the shape

$$f : (x : A) \rightarrow B(x), \quad \forall x, R(x, f(x)),$$

with $B(x)$ a data type. Instead of synthesizing f and proving the law afterward, synthesize

$$F : (x : A) \rightarrow \{y : B(x) \mid R(x, y)\}. \quad (2)$$

Algorithm 3 Native structural recursive-sketch generation

```

1: function STRUCTURALSKETCHES( $T, \Phi, \Gamma, b$ )
2:   Introduce the function telescope, preserving its dependencies
3:   for each relevant inductive argument  $x$  within the scheme budget do
4:     for each dependency-closed generalization set  $V$  within the motive budget do
5:       Build a native result family over  $x$ 's indices and  $V$ 
6:       Query the inductive/recursor metadata for branch telescopes
7:       for each reachable constructor branch do
8:         Introduce fields and checked recursive-result capabilities
9:         Specialize the behavioral relation to this branch
10:        Create coupled implementation and proof holes
11:      end for
12:      Submit the complete branch forest to the native scheduler
13:      On success, lower through the equation compiler and recheck the contract
14:    end for
15:  end for
16: end function

```

Set $f(x) = (F(x)).\text{val}$ and take $(F(x)).\text{property}$ as the proof. This is constructive: it does not extract data from a merely propositional existential using choice. It is especially effective for recursive functions because the recursive result carries both its value and the precise relation available to the next branch.

Proposition 10.1 (Pointwise packaging). *For the stated data-valued family, any term of (2) yields a solution of the original pointwise contract, and a solution f, p yields such a packaged function.*

Proof. The forward direction uses the two projections pointwise. The reverse direction maps x to $\langle f(x), p(x) \rangle$. Both are directly typed dependent functions; no inference from finite tests or noncomputable witness extraction is involved. $\square$

This transformation does *not* independently solve arbitrary global contracts. Injectivity, an involution law, or a law relating two different inputs may still depend on the entire function. A contract decomposer must supply a checked bridge from its local obligations to the original Φ . When it cannot, keep the global proof goal attached to the whole candidate.

10.4 Worked synthesis: relational list map

Let $\text{Aligned}(R, xs, ys)$ mean that xs and ys have the same list structure and their corresponding elements satisfy R . For a supplied $f : A \rightarrow B$, request

$$(xs : \text{List}(A)) \rightarrow \{ys : \text{List}(B) \mid \text{Aligned}(\lambda a b. b = f(a), xs, ys)\}.$$

The empty-input branch is forced by the relation's empty constructor. In the cons branch, recursive synthesis supplies

$$ys : \text{List}(B), \quad h : \text{Aligned}(\lambda a b. b = f(a), \text{tail}, ys).$$

The relation's cons constructor exposes an output head $b : B$ and the obligation $b = f(\text{head})$. The native application search proposes $f(\text{head})$, and reflexivity completes the local relation. The resulting output is $f(\text{head}) :: ys$.

This derivation uses types, constructor relations, and the available callback. It does not require an admitted `List.map` component. In the experiment, the author supplied the recursive skeleton and output-spine refinement, and the native solver filled the head and proof holes. The production algorithm

above is the proposed mechanism for discovering those sketches; that discovery is not implemented in the miniature prototype.

The length theorem follows from induction on `Aligned`. Crucially, the proof’s list indices should first be generalized to variables. Attempting induction directly on a relation whose second index is the compound expression `(listMap f xs).val` caused an actual failed experiment. A generic lemma over variable indices fixed it. This supplies a small but concrete regression test for the motive/generalization component.

10.5 Worked synthesis: indexed vectors

For a vector family $\text{Vec}(A, n)$, the type

$$(A \rightarrow B) \rightarrow \text{Vec}(A, n) \rightarrow \text{Vec}(B, n)$$

forces length preservation but does not fully specify elementwise mapping. In a nonempty branch it may allow repeating a transformed head rather than transforming each original position. Leant2 must distinguish the guarantee encoded by the index from additional semantics encoded by a relation or law.

The vector-map experiment checks that native branch synthesis can retain n correctly and construct a length-preserving recursive program from the supplied skeleton. A separate pointwise relation would be required to make it a verified implementation of a fully specified map operation. The actual generated branch is the expected mapping branch, but the requested bare vector type alone does not establish that characterization for all possible inhabitants.

For vector head, the type $\text{Vec}(A, n + 1) \rightarrow A$ permits elimination of the impossible empty case. Native indexed elimination produced a working implementation in the experiment. Its axiom inventory contained `propext`; this is an observed property of that generated elimination proof, not a reason to declare all indexed synthesis axiom-free or to prohibit indexed types.

10.6 Carrier invention: list indexing as a fold

A useful higher-order sketch for

$$\text{List}(A) \rightarrow \mathbb{N} \rightarrow \text{Option}(A)$$

folds the list into a *function-valued* accumulator. Let

$$C = \mathbb{N} \rightarrow \text{Option}(A), \quad z = \lambda i. \text{none},$$

and search for a step of type $A \rightarrow C \rightarrow C$. Its natural equations are

$$s(a, k, 0) = \text{some}(a), \quad s(a, k, n + 1) = k(n).$$

Then `foldr(s, z, xs)(i)` is the desired lookup function. The key search decision is the carrier C , not merely choosing a list constructor.

A practical carrier heuristic takes arguments that remain to be consumed after a traversal and abstracts them into the accumulator’s domain. The branch contracts then explain how those arguments change. This provides a specific algorithmic avenue for the kind of bounded-composition gap documented in the repositories, but it is not a claim to have solved their exact integer-indexed benchmark. Integer inputs require an explicit policy for negative indices, which must be included in the specification and proof rather than guessed.

10.7 Accumulator invariants and well-founded recursion

For an accumulator reverse, a strong candidate invariant is

$$\text{go}(xs, acc) = \text{reverse}(xs) ++ acc.$$

The top-level contract at $acc = []$ is too weak to justify a recursive call with a nonempty accumulator. A synthesis lane can propose invariants from relational summaries of admitted operations, small expression grammars, or user sketches. It must prove the initialization, branch preservation, and finalization implications. Failed proof search for one invariant does not refute the implementation.

For nonstructural recursion, enumerate a small family of measures: natural-valued arguments, list lengths, structural sizes, and lexicographic tuples. A proposed call on y from x creates the goal $\mu(y) < \mu(x)$ or the corresponding well-founded relation. Discharge it using hypotheses, simplification, arithmetic automation, or a supplied lemma. Retain it as an ordinary proof obligation until solved. Register the definition only after Lean accepts its termination elaboration [12].

This architecture also permits a relational recursion scheme where a smaller argument is returned together with a decrease proof. It avoids an unsafe shortcut: hypothesizing the desired recursive function at every input and later hoping that a termination checker accepts the resulting body. Recursive capabilities should constrain generation before final checking, while final checking remains authoritative.

11 The kernel/compiler boundary

11.1 A boundary established by experiment

The first native experiment defined vector map using an induction tactic inside an executable `def`. The resulting expression had the intended type and no axioms in its logical dependency inventory, but Lean’s code generator rejected the direct recursor use. Rewriting the same recursive sketch as pattern equations routed it through the equation compiler and produced executable code. The final program evaluated correctly.

This is a strong reason not to design Leant2 around the rule “an arbitrary accepted `Expr` can simply be emitted as an executable definition.” Lean elaboration and compilation are related but distinct stages [13]. Native semantic search should retain a structured recursion plan so that it can use the supported declaration and equation-compilation path.

11.2 A two-stage lowering protocol

For a recursion-free expression, closing and registering a normal definition may suffice. For a recursive plan, generate a declaration telescope, pattern equations, branch bodies, and any termination annotation through Lean’s declaration elaborator. The output of this stage is a fresh declaration bundle containing the main value and generated helpers.

The original logical plan and the compiled declaration need not be textually identical. There are two safe routes. The simplest is to prove Φ directly about the lowered declaration. Alternatively, prove an equality between the logical candidate and the lowered value, then transport the original proof. In both routes, the final certificate names the lowered value, not a pre-lowering placeholder.

The acceptance record should distinguish `kernelChecked`, `compiled`, and `evaluatedOnExamples`. Compilation is not a machine-checked theorem about the compiler’s native-code semantics. Evaluation tests are valuable regressions, not a replacement for the logical contract. A deployment requiring stronger end-to-end assurance needs an independently validated compilation story, beyond the scope of the current prototypes.

Algorithm 4 Lower and authorize a candidate bundle

```

1: function AUTHORIZE( $O, P$ )
2:   Restore the origin environment in an isolated validation context
3:   Lower the plan  $P$  to declarations through the relevant Lean elaborators
4:   Reject unresolved term/level metavariables, leaked free variables, and errors
5:   Kernel-check the declaration bundle and its exact contract theorem
6:   Audit transitive axioms and implementation dependencies against  $O$ 
7:   if the request requires executable code then
8:     Check the compilation policy and compile the lowered implementation
9:     Reject noncomputable or inadmissible runtime dependencies
10:  end if
11:  Export explicit replay source and check its elaborated result
12:  Publish the implementation, proof, and receipt atomically
13: end function

```

11.3 Partial and noncomputable definitions

The default implementation profile should reject candidates that require noncomputable choice or opaque partial recursion to meet an intended total executable interface. A theorem-proving profile can allow noncomputable objects under a declared foundation, but should label them accordingly. The existence of a type-correct opaque constant does not establish that its runtime implementation terminates or implements its expected equations.

Axiom auditing and executability auditing solve different problems. `Classical.choice` is relevant to mathematical assumptions and computability; an `extern` or an alternate runtime implementation can raise operational trust questions even without adding an obvious theorem axiom. Preserve a separate runtime-dependency inventory and state explicitly which runtime and compiler assumptions the output relies on.

12 Behavioral synthesis as coupled construction and proof

12.1 Three sources of useful constraints

The source type contributes structural information: vector lengths, subtype predicates, dependent output families, and interfaces that require callbacks or dictionaries. User examples contribute concrete observations. User laws and component summaries contribute logical relations. All three should participate in search, but they have different proof obligations.

Refinement-type synthesis shows the value of propagating a specification into subexpressions rather than treating verification only as a final filter [18]. Leant2 should adopt that principle without restricting every host proposition to a decidable refinement logic. A backend handles the fragment it understands; an opaque or higher-order property remains a genuine Lean goal for another lane or for user guidance.

A `ContractDecomposer` receives the native property and a proposed program constructor. It returns local obligations plus a reconstruction term showing that those obligations imply the original property. Initially implement a small library: conjunctions; pointwise universally quantified output relations; constructor-aligned relations; and registered component pre/postconditions. Avoid heuristic rewriting of arbitrary predicates into supposedly independent subgoals without a checked bridge.

12.2 Proof automation as a portfolio

Use lightweight kernel reduction and reflexivity first, then bounded simplification with controlled theorem sets, then targeted native arithmetic or congruence reasoning, and optional Aesop/Mathlib adapters. Induction and auxiliary lemma invention are separate strategies with explicit budgets. This layered approach makes inexpensive local obligations cheap without requiring every synthesis problem to load a large external library.

The default certificate profile should distinguish kernel reduction from native-code-backed decision shortcuts. A tactic that emits a proof can still depend on additional axioms. In particular, the name of a decision or certificate tactic does not alone establish that its checker is reduced inside the kernel. Audit the actual proof dependencies; do not describe a backend as axiom-free merely because its underlying solver produces a certificate [14, 11].

A proof of the contract’s negation is useful for rejecting a candidate. Failure to prove the contract, failure to prove its negation, or an automation timeout leaves the candidate unresolved. This preserves the predecessor’s positive/falsified/inconclusive distinction while permitting much stronger proof strategies [5].

12.3 Counterexample-guided synthesis with an unknown branch

Let $R(t, x)$ express the desired behavior on input x , and let E be the bank of accepted counterexamples. Candidate construction seeks a typed t satisfying the current finite obligations. Verification then attempts to establish $\forall x, R(t, x)$ or produce a concrete x for which $R(t, x)$ is false.

Algorithm 5 Fair CEGIS with proof-producing acceptance

```

1:  $E \leftarrow \emptyset; W \leftarrow \emptyset$  ▷ examples and unresolved candidates
2: while the global budget permits do
3:   Obtain a new typed candidate consistent with  $E$ , or revisit a candidate in  $W$ 
4:   Run a bounded verifier on the exact native specification
5:   if a proof of the specification is returned then
6:     Submit the candidate and proof to the common authorization gate
7:   else if a counterexample  $x$  is returned then
8:     Decode  $x$  into its native input type, including required preconditions
9:     Check a proof of the violation and add it to  $E$ 
10:    Repartition observation buckets and wake affected synthesis tasks
11:   else
12:     Retain the candidate in  $W$  with a larger future verification tier
13:   end if
14: end while
15: Report remaining candidates and bounded/inconclusive status

```

For a request with a precondition $Q(x)$, a returned counterexample must include evidence of $Q(x)$ as well as the violation. A solver model outside the dependent input domain is not a counterexample. For example, a model with an out-of-range integer is not an inhabitant of `Fin n`; a model of an abstract list length need not decode to a valid source list satisfying all path constraints.

The finite prototype implements the simple terminating case where all Boolean input pairs can be checked. The production design additionally requires the unknown branch, queues, proof-carrying counterexamples, and dependency-sensitive invalidation.

12.4 Finite domains and a genuine universal result

For a finite source domain, exhaustive checking can establish a universal property when the enumeration’s coverage is justified. In the experiment, Lean’s ordinary `decide` proves the property for both Boolean

arguments after the synthesized expression is quoted as a literal. The compiled search procedure is not trusted to establish the theorem. It merely chooses which small syntax tree the kernel then checks.

This is a useful architectural pattern: an efficient untrusted metaprogram proposes a small certificate-bearing artifact, while the final proof avoids replaying the expensive enumeration. The runtime of synthesis and the size of the proof are therefore decoupled. A similar boundary can support finite maps, small bit-vector grammars, or bounded expression templates, provided the source embedding and final proposition remain exact.

12.5 External solvers and arithmetic

An SMT adapter should explicitly identify its supported theory: for example, a quantifier-free integer arithmetic fragment with a particular interpretation of operations. A source `Nat` must not be translated as an unconstrained integer; natural subtraction must not silently become integer subtraction; bounded machine arithmetic must not silently become unbounded arithmetic. The adapter owns these translation lemmas.

A satisfiable result is a candidate model, not automatically a valid source witness. An unsatisfiable result is not a theorem unless a checked certificate and translation argument connect it to the exact Lean proposition. If reconstruction fails or an operator is unsupported, return `unknown`. A strict no-external-trust profile can use a solver solely to discover terms or counterexamples and ask native Lean tactics to prove their consequences.

For affine subproblems, a useful staged implementation is to enumerate a coefficient-bounded template, ask the solver for coefficients, substitute those coefficients into a native expression, and prove the requested law in Lean. The bounds are grammar restrictions. If the solver exhausts them, the result is template exhaustion, not global impossibility.

12.6 Higher-order observations and effects

For higher-order inputs, examples require explicit callable test values or symbolic specifications of callbacks. A finite collection of callbacks is not a universal quantifier over functions. Instead of manufacturing random closures and claiming functional coverage, expose an observation algebra: application at named arguments, constructor observations, and checked equality tests where decidable. Retain the limitations in the result.

Do not execute an arbitrary synthesized IO value merely to inspect its behavior. Start with pure models of state, exceptions, parsers, or finite traces; prove the implementation's relation to the model under explicit assumptions. Actual effectful testing requires an opt-in sandbox and an operational policy. A plain `IO Unit` signature does not specify which external actions are acceptable.

13 Guarantees, limits, and negative evidence

13.1 The success guarantee is small and strong

Theorem 13.1 (Conditional certificate soundness). *Assume the validation environment is the recorded environment, the kernel and imported foundation satisfy their stated trust assumptions, and the publication gate accepts only closed instances of (1) whose dependency inventories satisfy the request's policies. Every published `Verified` result inhabits the requested native type and satisfies the exact recorded behavioral predicate, relative to the displayed local premises and allowed axioms.*

Proof. The gate accepts the implementation declaration only after checking its value at the closed requested type. It accepts the correctness declaration only after checking its proof at the predicate applied to that

exact implementation. Closing and reopening the recorded telescope specializes those declarations to the original context. The dependency audit fixes which assumptions those judgments may use. Search order, examples, learned scores, solver answers, and abstract representations do not appear in this argument because none can authorize publication without the two checked judgments. $\square$

This theorem is an architectural implication, not a formal verification of the proposed gate’s implementation. The small accepted prototypes give examples of the relevant checks, not a proof that a future worker has no transaction or audit bugs. A valuable future formalization target is a small explicit checker for serialized candidate bundles and policy receipts, while the high-performance search remains untrusted.

13.2 Relative completeness is not a marketing claim

Fix a grammar $\mathcal{G}$ of candidate plans, a finite component alphabet, a policy, and an effective size measure. A plan includes any auxiliary definitions, recursion scheme, type instantiations, and proof certificate needed for acceptance. Assume there are finitely many plans of each size and a procedure that enumerates them. Invalid plans may fail or time out during elaboration; a valid accepted plan has a finite successful validation computation.

Theorem 13.2 (Fair relative discovery). *If a lane fairly enumerates all plans in $\mathcal{G}$ and dovetails their validation with unbounded increasing allowances, then every admissible plan with a finite successful validation computation is eventually accepted, unless the user stops the run.*

Proof. Let P be such a plan. Its finite encoding appears at a finite position in the enumeration. Its successful validation uses a finite amount of work, so some allowance tier suffices. Fair scheduling eventually visits that plan at that tier. Its validation then completes, and the common gate accepts it. $\square$

The qualifications matter. A heuristic provider cutoff, a permanently discarded observational bucket, a fixed universe bound, a restricted recursion library, or a proof procedure that never enumerates the needed proof can invalidate the relevant coverage premise. Practical interactive search has finite budgets and does not receive a termination or completeness guarantee from this theorem. The theorem explains how to preserve an honest completeness lane while optimizing other lanes aggressively.

A universal finite decider for all inhabited Lean types is not the goal. The implementable target is broad useful coverage with reliable positive certificates, explicit fragment guarantees, and informative inconclusive outcomes. The difficult research questions concern search efficiency and specification discovery, not permission to weaken the meaning of success.

13.3 Three different meanings of “no solution”

For a finite syntax grammar, enumeration can establish that no candidate in that grammar passes a complete checker. For a decidable logical fragment, a decision procedure can establish nonderivability in that fragment. A semantic Lean impossibility theorem is stronger and different: it supplies

$$q : (t : T) \rightarrow \Phi(t) \rightarrow \text{False}.$$

No scheduler flag can replace q .

A propositional LJT lane remains useful for fast inhabitation and fragment classification. The predecessor uses this tradition, originating in contraction-free intuitionistic sequent calculi [2, 17]. But an intuitionistic countermodel does not generally construct a Lean proof of the negation of the goal. With classical axioms available, a proposition such as $P \vee \neg P$ illustrates the difference particularly clearly. Even without classical axioms, failure to prove an arbitrary proposition is not a proof that it is false.

Therefore the negative ledger records the logic, grammar, assumptions, abstraction direction, completeness theorem, and exhausted bounds. A strict semantic `Impossible` result is reserved for a checked

source-level proof. The finite prototype contains a genuine such proof for the impossible specification $\forall b, f(b) \neq f(b)$; its native `noFalse` control, by contrast, records only that the bounded demo tactic did not solve `False`.

13.4 Sound rejection from abstractions

Proposition 13.3 (Completion-preserving pruning). *Suppose $A(P)$ overapproximates all behaviors of all grammar-admissible completions of a partial plan P , and the source contract permits only behaviors in B . If a checked argument establishes $A(P) \cap B = \emptyset$, pruning P preserves every satisfying completion in that grammar.*

Proof. Any satisfying completion would have behavior in $A(P)$ by overapproximation and in B by the contract. It would therefore supply an element of the supposedly empty intersection, a contradiction. $\square$

The overapproximation is over *completions*, not just the particular filled subterms sampled so far. Summaries that accidentally assume an unsolved callback is the identity, that ignore a constructor alternative, or that replace an opaque provider by a guessed length formula fail this premise. Make the premise an explicit adapter obligation.

13.5 Costs and minimality

Define a syntax cost before claiming minimality. For example,

$$c(t) = n_{\text{nodes}}(t) + w_c n_{\text{cases}}(t) + w_r n_{\text{recursiveCalls}}(t) + w_l n_{\text{nonlocalComponents}}(t).$$

This is not runtime complexity. A proof of an asymptotic resource bound needs a model of execution and a source theorem, possibly supplied by a resource-aware synthesis plugin. Prior resource-guided synthesis research offers a route for such an extension, but it should not be conflated with a tree-size heuristic [21].

The finite XOR experiment finds its first solution in an exact-size enumeration. This supports a minimality observation within that finite grammar and enumeration, and the included independent enumerator checks the same result. The Lean theorem in the artifact proves functional correctness, not the enumerator’s completeness or a minimality theorem. Keeping those claims separate makes the evidence more useful.

14 Production modules and APIs

14.1 Module boundaries

The following is a proposed source layout. The names specify responsibilities, not files already implemented in the prototype.

Module family	Owned responsibility
<code>Leant2.Frontend</code>	Tactic, command, and exploration syntax; request elaboration; user-visible diagnostics.
<code>Leant2.Core.Origin</code>	Frozen environment/context/specification identity and semantic policies.
<code>Leant2.Core.Transaction</code>	Version-adapted snapshots, owned assignments, context reinstallation, cancellation propagation.
<code>Leant2.Core.Plan</code>	Typed construction plans, recursive sketches, closure over telescopes, serialization and replay.

Module family	Owned responsibility
<code>Leant2.Search.Forest</code>	Residual goals, dependencies, blocked obligations, connected components, continuations.
<code>Leant2.Search.Native</code>	Introductions, exact use, application spines, constructors, projections, dependent elimination.
<code>Leant2.Search.Index</code>	Component metadata, native-head retrieval, widening tiers, optional abstract type graph.
<code>Leant2.Search.Schedule</code>	Work ledger, fair quanta, profiles, widening, cancellation, candidate and verifier queues.
<code>Leant2.Recursion</code>	Structural schemes, motive/generalization candidates, carrier invention, decrease obligations.
<code>Leant2.Contract</code>	Exact property attachment, proved decomposition rules, example bank, relational summaries.
<code>Leant2.Backend</code>	Optional finite grammar, arithmetic, SMT, learned proposals, and abstract-interpretation adapters.
<code>Leant2.Proof</code>	Core proof tactics and optional pinned Aesop/Mathlib strategies; proof budgets and evidence.
<code>Leant2.Check</code>	Closing, kernel checking, axiom audit, dependency audit, execution policy, final receipts.
<code>Leant2.Emit</code>	Equation-compiler lowering, explicit source export, replay, and atomic declaration publication.
<code>Leant2.Worker</code>	Project-local process protocol, imported environment lifecycle, isolated expensive tasks.

Separate the optional Mathlib and external-solver adapters into distinct packages. The native core and basic examples should depend only on the selected Lean distribution. This reduces cold-start cost and makes version mismatches easier to diagnose. Aesop and Mathlib adapters must be pinned to compatible versions rather than loaded from a floating branch into an arbitrary project.

14.2 Capability-oriented plugin interfaces

A proposal lane may inspect a goal and return a plan, but it may not publish declarations. A proof backend may return a proof expression, but it may not change the origin's axiom policy. A verifier may return a model, but it may not insert it into the counterexample bank until native validation succeeds. These capabilities should be distinct interfaces rather than conventions inside one large monad.

A schematic API is:

```

Proposallane.step : Origin -> Task -> Quantum -> WorkerM StepResult
ContractDecomposer.open : Goal -> ConstructorPlan -> WorkerM Decomposition
CounterexampleChecker.check : Origin -> Candidate -> Model -> WorkerM CheckedCex
Lowerer.lower : Origin -> ClosedPlan -> WorkerM DeclarationBundle
CandidateGate.validate : Origin -> DeclarationBundle -> WorkerM Verdict
Publisher.commit : AuthorizedBundle -> CommandElabM Publication

```

These signatures are design pseudocode. In a concrete implementation, `AuthorizedBundle` and `CheckedCex` should have private constructors and origin-indexed handles. Their contents include the exact expressions and policy evidence, not just a success Boolean. A serialized handle must be revalidated before it acquires authority in another process.

14.3 Native API adapters to implement first

The experiment verified the existence and use of the following Lean 4.34.0 API surface: `Meta.saveState` and `restore`, `MVarId.intro1P`, `MVarId.apply`, `MVarId.cases`, `MVarId.induction`, fresh opaque goal creation, `Meta.check`, and `Lean.addDecl`. Only the subset visible in `NativeSearch.lean` is used in its synthesis loop. The initial API probe checked the other signatures, not a finished implementation around them.

Build wrappers for: exposing a goal telescope; creating owned holes; exact-context constant instantiation; native unification with rollback; retrieving inductive metadata; dependent elimination; closing an expression; declaration checking; and collecting transitive dependencies. These wrappers should centralize version sensitivity. The search layer should not scatter raw elaborator state mutations across dozens of modules.

14.4 Do not overload a tactic interface with declaration synthesis

A tactic can fill a term hole with a nonrecursive expression or an application of existing recursive combinators. Creating a new recursive definition, helper theorem, and executable declaration bundle naturally belongs at the command-elaboration level. The tactic interface should either use an internal local helper construction that the elaborator explicitly supports, or delegate to a declared helper-generation service with an auditable scope.

This avoids a common confusion: a tactic producing a term by induction can satisfy the kernel’s logical expectations without arranging the declarations and compiler metadata needed for execution. The command frontend is not optional cosmetic syntax once recursive code generation is a central feature.

15 Publication, provenance, and reproducible output

15.1 What the final receipt records

A receipt records the exact requested type and property, the closed implementation and proof identities, environment and toolchain identities, admitted components, axiom inventory, local premises, execution status, and the budgets actually consumed. It also records whether the implementation was generated from scratch, selected as an existing component, or completed from a user or system sketch.

A source hash is useful for connecting a remote check to a local file, but it is not a proof certificate by itself. The authoritative evidence is the checked declaration bundle in its recorded environment. A receipt should not claim cryptographic attestation unless the service actually provides one. The experiment summaries in this archive are explicitly transcribed tool-result summaries, not signed service attestations.

15.2 Exact-source replay

Pretty printing may omit implicit arguments or choose notation that elaborates differently in another scope. Export sufficiently explicit names, universe arguments, binders, and helper declarations. Re-elaborate that exported source in a fresh validation context and compare the resulting candidate with the authorized native value under the declared comparison policy. If direct equality is inconvenient, prove the contract afresh for the exported declaration.

Do not assume that two identical strings have interchangeable typed provenance. A local identifier may resolve to different constants or dictionaries, and a declaration’s universe instantiation may differ despite similar text. Preserve the source owner and origin through normalization, display deduplication, and selection. This retains a soundness discipline already prominent in the predecessor’s internals [4].

15.3 Atomic environment mutation

Publishing a recursive implementation can add auxiliary definitions, equation theorems, and a correctness theorem. Stage the complete bundle in an isolated environment. If compilation, proof checking, policy audit, or source replay fails, publish nothing. On success, add the bundle in dependency order and record a single publication transaction.

Environment caches should use persistent revisions. If an imported module, local instance, component registration, or semantic option changes, invalidate affected queries. A proof of an unchanged-looking proposition in an earlier environment is not automatically a proof under the new meaning of all referenced names. Stable hashes and exact dependency tracking make invalidation selective without making it speculative.

16 Operational safety and resource behavior

Use three independent controls: logical work budgets, wall-clock deadlines, and process-level resource limits. Heartbeats are useful but do not account for every external computation, compilation step, or IO wait. A deadline needs cancellation that actually stops the relevant worker or process tree. Solver timeouts must not leave stale responses that can be attached to the next request.

A task scheduler should produce periodic structured progress: active lane, explored plans, verified candidates, disproved candidates, pending proofs, memory use, and the reason for a pause. Avoid overwhelming the user with every unification failure. A compact failed-attempt explanation might say: “Both list constructors were considered; the cons branch is waiting for a recursive result at a smaller index; no applicable recursion scheme is enabled.”

Persist only closed plans and origin metadata across worker restarts. A raw `MVarId` from a dead process has no usable authority. In-memory snapshots can be efficient and persistent, but their lifetime is the owning environment and toolchain. A long-lived daemon should separate project identities and avoid silently reusing declarations compiled against a different Lean release.

The resource ledger should separately count synthesis and proof work. A candidate that is easy to generate but expensive to verify may be less useful interactively than a slightly larger program with a short certificate. The ranking model can learn that preference, but the completeness lane must not permanently exclude expensive proofs solely because an early estimate was pessimistic.

17 Practical coverage and deliberate boundaries

Family	Principal algorithms	Initial status target
Arrows and higher-order plumbing	Native introductions, exact reuse, application spines	Core milestone
Products, sums, records	Constructor search, projections, case analysis	Core milestone
Dependent pairs and subtypes	Coupled witness/proof forest, relations	Core milestone
Finite indexed families	Native index refinement, dependent elimination	Early extension
Polymorphic and universe-polymorphic callbacks	Exact telescopes and levels; bounded type invention	Early extension
Type-class-based components	Native instance synthesis plus explicit dictionary identity	Early extension

Family	Principal algorithms	Initial status target
Lists and ordinary trees	Registered folds; generated structural sketches	Recursion milestone
Accumulator algorithms	Carrier invention, invariant templates, strengthened motives	Advanced extension
Well-founded or mutual recursion	Explicit schemes and checked decrease obligations	Advanced extension
Quotients and abstract mathematics	Library reuse, lifting proofs, permitted noncomputable profile	Selective support
Type-valued programs	Bounded native type-expression synthesis and recursion	Explicit opt-in grammar
Effectful programs	Pure models, trace specifications, optional sandboxed evaluation	Separate policy layer
Arbitrary theorems or invariants	General proof search and optional learned proposals	No uniform coverage claim

The phrase “a large fraction of types that occur in practice” needs a workload definition. A corpus of data-transformation APIs has different needs from a corpus of algebraic theorems or effectful systems code. Report coverage by family, with exact imports, provider inventories, contracts, and sketch assistance. A single overall percentage conceals more than it explains.

The strongest practical leverage is likely to come from combining component reuse, native dependent constraints, and a modest library of verified recursion schemes. This is an engineering hypothesis, not a measured result of the present experiments. The benchmark plan below is intended to test it rather than assume it.

18 Executed Lean experiments

18.1 Method and reproducibility boundary

The local document-building environment had no Lean toolchain and could not resolve external hosts. The supplied `tools.zip` described the AXLE check API and supplied Python and Wolfram Language clients. A Wolfram evaluator was able to send UTF-8 JSON requests to that service, so the Lean experiments were actually compiled remotely rather than merely inspected.

Both final files were submitted to environment `lean-4.34.0` with `ignore_imports = false`, `theorems_only = false`, and a 40-second server allowance. Each imports only Lean. The requests checked the whole file, including definitions, theorems, `##eval` commands, and `##print axioms` commands. The reported version was 4.34.0. Source hashes matched the files included in this archive, and the echoed source matched after trimming surrounding whitespace.

The service’s `okay` field was not used alone. Acceptance additionally required no failed declarations, no Lean or service errors, no sorry-use diagnostics, and the expected axiom reports. This matters because a service can report successful processing for incomplete proof material. The service’s published checking interface and the user’s clients both make the distinction relevant [22].

The native file produced two unused-tactic linter warnings for its `fail_if_success` negative controls. Both files received service advisories about not using its cached Mathlib header; the import-preservation check showed that the requested core-only headers remained intact. No local Lean compilation, independently installed second Lean kernel, or signed remote attestation is claimed. The archive includes summaries of the tool responses, exact submitted files, and scripts for independent reruns.

Item	Native search	Finite CEGIS
File	<code>NativeSearch.lean</code>	<code>FiniteCEGIS.lean</code>
Source lines	219	114
Lean version	4.34.0	4.34.0
Final service verdict	<code>okay = true</code>	<code>okay = true</code>
Failed declarations	None	None
Lean errors	None	None
Axiom reports	24	5
Empty inventories	23	5
Nonempty inventory	<code>vectorHead: propext</code>	None
Server-reported total	1610 ms	1574 ms

Table 5: Actual final runs. Times describe these service runs, not isolated synthesis timings or a benchmark comparison.

The documentation consulted is rolling documentation: during retrieval it identified a later release-candidate manual. API compatibility claims in this article are therefore anchored to the actual 4.34.0 experiments, not to an assumption that every rolling-reference feature is present in that environment.

18.2 Native search: what was exercised

The native solver uses introductions, local application, target constructors, reflexivity, and dependent case analysis under a depth bound of 80. Its essential feature is that a choice remains backtrackable until its whole residual forest succeeds. It has no global library search, no learned ranking, no CEGIS loop, and no automatic recursive-sketch generator.

Declarations or group	What the run establishes
<code>identity</code> , <code>compose</code> , <code>substitute</code>	Native polymorphic and higher-order local composition.
<code>swap</code> , <code>mapSum</code>	Product construction and sum elimination.
<code>dependentApply</code> , <code>dependentPair</code> , <code>dependentProjection</code>	Real dependent argument and pair types, not erased approximations.
<code>transport</code>	Equality-dependent movement between fibers.
<code>rankN</code> , <code>mixedUniverses</code>	Polymorphic arguments and two independently scoped universe-polymorphic callbacks. Not the repositories' full two-universe Church benchmark.
<code>chooseSecond</code>	A later behavioral equality causes backtracking over an earlier function choice.
<code>chooseSecond_all</code>	A separate proof of the stronger universal law for the generated selector.
<code>exactDictionary</code>	Distinct explicitly supplied record values remain distinguishable. Not a new type-class solver.
<code>emptyElim</code> , <code>vectorHead</code>	Empty elimination and a real indexed impossible branch.
<code>rollback</code>	A successful metavariable assignment disappears after restoring the checkpoint.
<code>noFalse</code> , <code>universeNegative</code>	Expected failure of a bounded search and rejection of an invalid universe assignment.
<code>vectorMap</code>	Synthesized branches inside an author-supplied structural recursion skeleton.

Declarations or group	What the run establishes
<code>listMap</code> , length lemmas	Relational hole filling inside an author-supplied recursive/output-spine sketch, plus a hand-written general length proof.

Actual evaluations returned 29 for the numeric selector, "right" for its string instance, [11, 12, 13] for the relational list transformation, 7 for vector head, and [11, 12] for the vector transformation followed by conversion to a list. The source contains the exact evaluation inputs and all audited declaration names.

The solver's blanket exception catch, fixed depth bound, repeated case-split enumeration, and lack of goal indexing are deliberate prototype limitations. They are not recommended production behavior. The generated final declarations, however, do not depend on the search metaprogram or on its partial recursion: the search is an untrusted construction procedure whose output is ordinary Lean code and proofs.

18.3 Finite CEGIS: exact grammar and trace

The grammar is

$$e ::= x \mid y \mid \neg e \mid (e \wedge e) \mid (e \vee e),$$

with every syntax-tree node costing one. Let a_n be the number of expressions of exactly n nodes, with $a_0 = 0$. Structural decomposition gives

$$a_1 = 2, \quad a_n = a_{n-1} + 2 \sum_{k=1}^{n-2} a_k a_{n-1-k} \quad (n \geq 2).$$

The factors correspond to one unary constructor and two ordered binary constructors. The implementation enumerates those cases directly, without commutativity deduplication. The observed counts were

$$(a_0, \dots, a_8) = (0, 2, 2, 10, 26, 114, 402, 1722, 6890),$$

for 9168 candidates up to eight nodes.

The desired truth function is $x \neq y$. Starting with no examples, the five proposals and their counterexamples were:

Round	Proposal	New counterexample (x, y)
1	x	$(0, 1)$
2	y	$(1, 0)$
3	$x \vee y$	$(1, 1)$
4	$\neg(x \wedge y)$	$(0, 0)$
5	$(x \vee y) \wedge \neg(x \wedge y)$	None

The final expression has eight nodes. A term elaborator quotes that literal expression into Lean, and ordinary `decide` proves

```
∀ x y : Bool, synthesizedXor.eval x y = (x != y)
```

The generated syntax and this correctness theorem have empty axiom inventories. The other three audited results establish an observational collision, its separation at a later input, and a genuinely impossible specification.

The Python reference script independently reproduced the counts, proposal trace, truth table, and absence of a smaller solution in the enumerated grammar. It is not a second Lean checker. The Lean enumeration uses repeated list concatenation and retains every syntax tree; a production finite backend should use arrays or builders, structural sharing, and exact finite-domain truth-table indexing where justified. No efficiency claim is made for this toy enumerator.

18.4 Failed attempts are part of the evidence

The archive retains three earlier failing source files under `evidence/`, outside the build targets. The first native attempt exposed the recursor/code-generation boundary, mistakenly used `theorem` for a data-producing sort-polymorphic declaration, and referred to a relation not present in the chosen core-only import. The second native attempt fixed those points but attempted induction with a compound relation index. The initial finite attempt used variable names colliding with grammar constructor names in a pattern definition.

These were genuine errors, corrected before the final successful runs. In particular, diagnostic pretty-printing of an intermediate term is not evidence that its whole file compiled: earlier error recovery produced `sorryAx` dependencies in incomplete declarations. The final acceptance table excludes those runs. Retaining them documents why checking only the successful-looking declarations or only an okay field would be inadequate.

19 Benchmark plan and development gates

19.1 A benchmark that measures the intended system

Construct a versioned corpus with separate strata for simple types, dependent pairs, indexed inductives, polymorphic callbacks, local dictionary identity, recursive traversals, accumulator algorithms, and explicit negative controls. Every entry records its Lean type, exact property, imports, admitted implementation components, proof-only components, permitted axioms, resource profile, and any supplied sketch.

Keep reference implementations and test oracles out of the implementation-provider inventory. Proofs may legitimately refer to a reference specification, but the candidate's computational dependencies should be audited separately to ensure it did not simply call the answer. An opaque wrapper around a forbidden provider is still leakage if the benchmark's policy prohibits it transitively.

Separate three assistance levels: no sketch, a supplied recursion skeleton, and a supplied output/invariant sketch. A system solving a sketched map task has not demonstrated automatic recursion discovery. Similarly, solving a specialized monomorphic task is not success on the original universe-polymorphic task. Compare exact requested declarations, not nearby convenient variants.

19.2 Metrics and ablations

Report time to first verified executable result, total raw proposals, native rule work, peak live search memory, contract-proof work, candidate size, proof size, and remaining unknowns. Record compilation and source-replay failures separately from type or behavior failures. Present distributions and censored timeouts rather than an average over successes alone.

The important ablations are structural: native dependency retention versus opaque indexed atoms; whole-forest versus greedy child commitment; contracts during construction versus postfiltering; component retrieval versus local-only search; recursive-scheme generation versus supplied sketches; heuristic-only versus fairness-preserving scheduling; and reversible versus irreversible observation bucketing. Some ablations are intentionally incorrect and belong in regression tests, not in production configuration.

For predecessor comparisons, pin both repositories, toolchains, fixtures, and all limits. The current repository documentation contains historical acceptance ledgers; those should not be reinterpreted as a fresh end-to-end pass rate. The present report did not execute Leant or Djex, so it supplies no comparative speedup or coverage number.

19.3 Milestone gates

Gate A: semantic core. Implement origin capture, native forest transactions, simple introductions/applications/elimination, and final checking. Require rollback, scope, duplicate-name, universe, and false-success regressions before adding broad search. The native prototype is a seed for this gate, not its completed deliverable.

Gate B: practical component synthesis. Add indexed retrieval, exact provider occurrences, ordinary native class synthesis, type-invention tiers, and a fair scheduler. Require replay of generated code in fresh scopes and preservation of distinct same-typed dictionaries. Measure retrieval misses explicitly.

Gate C: contract integration. Add exact property attachment, the pointwise packaging transform, proved constructor decomposition, a counterexample bank, and separate verification queues. Require finite observations never to become an unstated universal guarantee and require inconclusive verification not to suppress future candidates.

Gate D: recursive executable synthesis. Generate structural sketches, select dependency-closed motives, lower through the equation compiler, and attach relational proofs. Require no-sketch list and tree tasks, indexed-family regressions, and compiler-boundary tests. Only after this gate should the system be described as an automatic recursive implementation synthesizer.

Gate E: broader search. Add accumulator carriers, invariant grammars, well-founded schemes, optional solver adapters, and proof-aware optimization. Require every adapter to document its fragment, translation, proof reconstruction, failure semantics, and resource behavior. Learned proposals may be introduced here or earlier, but should never weaken the candidate gate.

These are acceptance gates, not delivery dates. A phase should advance because its correctness and coverage evidence is convincing, not because a predefined calendar expired.

20 Final design choices

Leant2’s most important decision is to make a native Lean problem, with all of its dependencies and assumptions, the permanent object of search. The most important algorithmic decision is to let implementation choices and later proof obligations backtrack together. The most important practical extension is a library of native recursive sketches whose branches are synthesized under relational contracts and whose final code passes through Lean’s executable declaration pipeline.

The system should borrow methods rather than inherit restrictions: logical focusing from inhabitation engines, rule indexing and structured proof search from Lean automation, refinement propagation from synthesis research, and counterexample-guided search for tractable sublanguages. Each method remains subordinate to the exact source type and property.

The experiments establish that small Lean-native mechanisms already handle genuine dependencies, universe-polymorphic callbacks, shared witness/proof choices, and a finite synthesis loop with a universal certificate. They also expose a boundary that a paper design could easily miss: logical acceptance and executable code generation require separate validation. Building that distinction into the architecture now is more valuable than adding it as an exception after a large search engine has been written.

A Artifact inventory and rerunning the experiments

The archive contains the article’s master `Leant2.tex`, section sources, and PDF; the two final Lean files; a core-only toolchain file and local build script; an independent Python enumeration; an exact-source remote-check script; the original user-provided AXLE clients; and evidence summaries plus earlier failed experiments. Failed sources are not build targets.

For an existing Lean 4.34.0 installation, run from the archive root:

```
./scripts/check_local.sh
python3 scripts/finite_reference.py
```

The local script checks the two standalone files individually; no Mathlib package is required. The supplied `lean-toolchain` selects `leanprover/lean4:v4.34.0`. Installing a toolchain may require network access and is outside the local checks actually performed for this report.

The remote script submits the exact files and audits every printed declaration:

```
python3 scripts/check_remote.py prototype/NativeSearch.lean
python3 scripts/check_remote.py prototype/FiniteCEGIS.lean
```

This script requires network access to AXLE. Its default axiom allowance is only `propext`, enough for the observed indexed elimination; all other audited outputs were axiom-free. It writes a complete new response receipt for the rerun. The evidence distributed with this article is explicitly a transcription of the connector’s observed result fields, not a fabricated original HTTP response file.

Build the article with:

```
cd article
latexmk -pdf -interaction=nonstopmode -halt-on-error Leant2.tex
```

The reproduction scripts were syntax-checked locally. The Python finite reference was executed locally. The two final Lean sources were checked remotely. The local shell script and the new Python remote wrapper were not themselves used to perform those Lean checks; their commands encode the same standalone inputs and validation policy.

B A compact implementation checklist

Before returning a candidate, ask whether the exact type and constraint were elaborated in the intended scope; whether external metavariables stayed rigid; whether all internal holes are closed; whether universe and dictionary assignments remained correlated; whether all branch obligations and termination proofs were discharged; whether the final lowered declaration rather than a precursor owns the proof; whether its axioms and runtime dependencies satisfy policy; and whether the exported spelling was replayed.

Before pruning a branch, ask whether the reason is a genuine contradiction, a checked abstraction rejection, or merely bounded failure; whether the reason depends on examples, local instances, path conditions, or a particular provider specialization; whether those dependencies are in the cache key; and whether later evidence can split a provisional observational bucket.

Before claiming coverage, ask whether the task was the original type, at its original universes, with its full property; whether the reference implementation leaked into providers; whether a recursion or invariant sketch was supplied; whether timeout cases were retained in the denominator; and whether the claimed result was typed, behaviorally verified, executable, or all three.

References

- [1] Vladimir Reshetnikov. *Leant*, README and repository documentation. Snapshot 6bf05ad78c467989e68290f2d08bbbed40802d485, inspected September 17, 2026. <https://github.com/VladimirReshetnikov/Leant/tree/6bf05ad78c467989e68290f2d08bbbed40802d485>.
- [2] Vladimir Reshetnikov. *Djex*, README and repository. Snapshot e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5, inspected September 17, 2026. <https://github.com/VladimirReshetnikov/Djex/tree/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5>.
- [3] Leant contributors. *Native source-to-fragment translation*, `src/Leant/Synth/Fragment.hs`, lines 1–210 of the inspected snapshot. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/src/Leant/Synth/Fragment.hs>.
- [4] Leant contributors. *Synthesis internals*, inspected opening sections and semantic-origin boundaries. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/synth-internals.md>.
- [5] Leant contributors. *Behavioral assertions during synthesis*, inspected lines 1–210. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/behavioral-synthesis.md>.
- [6] Leant repository report. *Leant / Djex Lean 4 rewrite analysis*, August 15, 2026; inspected abstract and opening material. Its own baseline predates the snapshots used in this article. https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/Leant_Djex_Lean4_Rewrite_Analysis/Leant_Djex_Lean4_Rewrite_Analysis.tex.
- [7] Lean FRO and Lean contributors. *The Lean Language Reference: The Type System*. Rolling reference, accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The-Type-System/>.
- [8] Lean FRO and Lean contributors. *The Lean Language Reference: Universes*. <https://lean-lang.org/doc/reference/latest/The-Type-System/Universes/>.
- [9] Lean FRO and Lean contributors. *The Lean Language Reference: Inductive Types*. <https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/>.
- [10] Lean FRO and Lean contributors. *The Lean Language Reference: Type Classes*; and *Init.Prelude*, output and semi-output parameter documentation. <https://lean-lang.org/doc/reference/latest/Type-Classes/>. <https://lean-lang.org/doc/api/Init/Prelude.html>.
- [11] Lean FRO and Lean contributors. *The Lean Language Reference: Tactic Reference*. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [12] Lean FRO and Lean contributors. *The Lean Language Reference: Recursive Definitions*. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [13] Lean FRO and Lean contributors. *The Lean Language Reference: Elaboration and Compilation*. <https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/>.
- [14] Lean FRO and Lean contributors. *The Lean Language Reference: Axioms; Validating a Lean Proof*. <https://lean-lang.org/doc/reference/latest/Axioms/>. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [15] Jannis Limperg and contributors. *Aesop: White-box automation for Lean 4*, repository README. Accessed September 17, 2026. <https://github.com/leanprover-community/aesop>.
- [16] Jannis Limperg and Asta Halkjær From. *Aesop: White-Box Best-First Proof Search for Lean*. CPP 2023. DOI: 10.1145/3573105.3575671. Author-deposited preprint metadata: <https://zenodo.org/records/7430233>.

- [17] Roy Dyckhoff. *Contraction-Free Sequent Calculi for Intuitionistic Logic*. Journal of Symbolic Logic 57(3), 795–807, 1992. This is the historical calculus lineage, not a claim that the prototype implements the full calculus.
- [18] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. PLDI 2016; preprint arXiv:1510.08419. <https://arxiv.org/abs/1510.08419>.
- [19] Jeevana Priya Inala, Nadia Polikarpova, Xiaokang Qiu, Benjamin S. Lerner, and Armando Solar-Lezama. *Synthesis of Recursive ADT Transformations from Reusable Templates*. 2015. arXiv:1507.05527. <https://arxiv.org/abs/1507.05527>.
- [20] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. *Program Synthesis by Type-Guided Abstraction Refinement*. 2019. arXiv:1911.04091. <https://arxiv.org/abs/1911.04091>.
- [21] Tristan Knoth, Di Wang, Nadia Polikarpova, and Jan Hoffmann. *Resource-Guided Program Synthesis*. 2019. arXiv:1904.07415. <https://arxiv.org/abs/1904.07415>.
- [22] Axiom Math. *AXLE documentation: check*. <https://axle.axiommath.ai/v1/docs/tools/check/>. The request/response validation protocol was additionally informed by the user’s supplied `tools.zip`. Actual experimental evidence is identified in the archive’s `evidence/` directory.